MULTI-DISASTER EARLY WARNING SYSTEM

Hardware-to-Dashboard Connection & Deployment Manual • Comprehensive Integration Guide

Target System: ESP32 + GPS (NEO-6M) + Sensors	**Active Server IP:** 10.166.170.10:8000
Transport Protocol: HTTP REST over 2.4GHz Wi-Fi	**Endpoint:** /api/v1/telemetry
Supported Hazards: Wildfire, Flash Flood, Landslide	**Dashboard URL:** http://localhost:8000

1. System Architecture & End-to-End Data Path

The system operates across three independent physical transmitter systems communicating with a central software receiver over a shared Wi-Fi network. Each node transmits its unique hardware identifier, disaster category, real-world satellite GPS coordinates, and raw sensor readings.

[PHYSICAL TRANSMITTER NODE]
- ESP32 Microcontroller
- Ceramic Patch GPS Antenna
- Dedicated Environmental Sensors
- Connects to 2.4GHz Wi-Fi

■■■■■■■
HTTP POST
JSON Packet
(Port 8000)
■■■■■■■

[CENTRAL RECEIVER & DASHBOARD]
- Python FastAPI Asynchronous Receiver
- Independent Classification Engines
- NASA FIRMS Satellite Cross-Check (+15%)
- Real-Time Leaflet OpenStreetMap Visualizer

2. Step-by-Step Connection Procedure

Step 1: Verify Host Computer Network Address

Your Mac hosts the central server. The ESP32 requires the Mac's private local IP. In Terminal, run: ipconfig getifaddr en0. Your currently assigned IP is **10.166.170.10**. If your Wi-Fi changes, always re-check this address.

Step 2: Ensure Dashboard Server Is Active

Launch the receiver from the project root:
cd /Users/aniruthreddy/Downloads/jarvis && ./venv/bin/python3 run.py
Confirm the server prints: *Uvicorn running on http://0.0.0.0:8000*. Navigate to **http://localhost:8000** in your web browser.

Step 3: Non-Invasive ESP32 Firmware Integration

Do **not** overwrite your existing sensor algorithms or trained machine learning models. Simply insert the following components into your current Arduino sketch:

```
// --- 1. HEADERS & CONFIGURATION (Insert at top of .ino file) ---
#include <WiFi.h>
#include <HTTPClient.h>
const char* WIFI_SSID = "YOUR_WIFI_NAME";
const char* WIFI_PASSWORD = "YOUR_WIFI_PASSWORD";
const char* SERVER_URL = "http://10.166.170.10:8000/api/v1/telemetry";

// --- 2. SETUP CODE (Add to void setup) ---
WiFi.begin(WIFI_SSID, WIFI_PASSWORD);
while (WiFi.status() != WL_CONNECTED) { delay(500); Serial.print("."); }
Serial.println("\n[Wi-Fi] Connected to Dashboard!");

// --- 3. TRANSMISSION FUNCTION (Paste at bottom of file) ---
void sendDisasterAlert(float lat, float lon, float temp, float smoke) {
  if (WiFi.status() == WL_CONNECTED) {
    HTTPClient http;
    http.begin(SERVER_URL);
    http.addHeader("Content-Type", "application/json");
    String payload = "{\"device_id\":\"FIRE_001\",\"hazard_type\":\"WILDFIRE\","
      "\"latitude\":" + String(lat, 6) + ",\"longitude\":" + String(lon, 6) + ","
      "\"sensor_data\":{\"temperature\":" + String(temp, 1) + ",\"smoke\":" + String(smoke, 1) +
",\"flame_detected\":true}}";
    int code = http.POST(payload);
    Serial.println("[TX] Response: " + String(code));
    http.end();
  }
}
```

3. Physical Hardware Pinout & Wiring Table

Correct pin pairing between the ESP32 and peripheral sensors is critical. Note that the GPS Transmit (TX) line must cross over to the ESP32 Receive (RX) pin.

Module / Component	Module Pin	ESP32 Target Pin	Functional Description & Notes
NEO-6M GPS Module	VCC	3.3V or 5V	Power input (recommended 3.3V to minimize thermal drift)
NEO-6M GPS Module	GND	GND	Common system ground
NEO-6M GPS Module	TX	**GPIO 16 (RX2)**	**Data Out from GPS → Into ESP32 Hardware Serial2**
NEO-6M GPS Module	RX	GPIO 17 (TX2)	Command input into GPS (optional for standard NMEA read)
MQ-2 / MQ-135 Gas	A0 (Analog)	GPIO 34 (ADC1)	Smoke density measurement (ADC1 active during Wi-Fi)
Flame Sensor	D0 (Digital)	GPIO 25	Binary optical flame detection trigger
HC-SR04 Ultrasonic	TRIG / ECHO	GPIO 5 / GPIO 18	Flood stage distance measurement
Soil Moisture Sensor	AOUT	GPIO 35 (ADC1)	Landslide soil water saturation percentage

4. Concentric Locality Risk Radius Classification

When sensor readings transition from **CLEAR** to **DANGER**, the dashboard automatically centers on the satellite GPS coordinates and renders 3 concentric risk zones based on the computed composite risk percentage:

Classification Zone	Map Visual	Evacuation & Public Action	Formula & Example (R=85%)
■ **Zone 1: VERY DANGER**	Crimson Pulse Fill: 38%	**Immediate Mandatory Evacuation**. Severe threat to life, active fire/flood front.	r = (R/100) × 1.5 km **< 1.28 km**
■ **Zone 2: DANGER**	Orange Dashed Fill: 22%	**High Warning Zone**. Prepare evacuation, clear critical routes, shelter vulnerable.	r = (R/100) × 3.5 km **< 2.98 km**
■ **Zone 3: MODERATE DANGER**	Amber Dotted Fill: 12%	**Advisory & Watch Zone**. Monitor emergency broadcasts, prepare go-bags.	r = (R/100) × 6.0 km **< 5.10 km**

5. Diagnostic & Troubleshooting Matrix

Every anticipated hardware, network, and software error condition is categorized below along with its immediate physical resolution.

Symptom / Error Code	Root Cause	Required Physical / Software Action
Wi-Fi dots loop indefinitely (Connecting.....)	ESP32 attempting connection to a 5GHz network band.	ESP32 hardware contains a 2.4GHz radio only. Switch your router or phone hotspot to **2.4GHz** (enable 'Maximize Compatibility' on iPhone). Verify SSID spelling.
HTTP POST Error: -1 (Connection Refused)	Subnet mismatch or server unreachable.	Verify that both your Mac and ESP32 share the same Wi-Fi. Check that your Mac's IP matches 10.166.170.10. Check that Python server is running on port 8000.
GPS Coordinates Show 0.000000, 0.000000	No satellite lock acquired yet.	Place the ceramic square patch antenna with the **flat tan side facing straight UP** outdoors or directly on a windowsill. Wait 60–90 seconds until the module LED blinks.
Brownout Detector Was Triggered	Power surge voltage drop when Wi-Fi modem engages.	Wi-Fi transmission requires ~250mA instant current. Power the ESP32 from a USB 3.0 port or dedicated 5V 1A power supply. Do not power through high-resistance unpowered hubs.
WiFi.h: No such file or directory	Incorrect microcontroller target board in IDE.	Navigate to *Tools* → *Board* in Arduino IDE and select **ESP32 Dev Module**.
HTTP Status Code: 400 (Bad Request)	Malformed JSON telemetry string.	Ensure JSON field keys match standard schema: device_id, hazard_type, latitude, longitude, sensor_data.
Node Collisions On Shared Wi-Fi	Concern over simultaneous transmissions.	No collision can occur. Wi-Fi uses hardware CSMA/CA packet scheduling and the Python FastAPI backend is asynchronous (asyncio). All 3 nodes can transmit simultaneously.

6. Telegram Early Warning Bot Setup Guide

Emergency dispatches and locality evacuation orders are transmitted in real-time to Telegram responders and municipal groups. Follow these steps to connect your bot:

Step	Action	Exact Command / Procedure
1. Create Bot	Open Telegram and search for **@BotFather**	Send /newbot, specify your display name and unique bot username ending in 'bot'.
2. Copy Token	Obtain the HTTP API Bot Token	Copy the string (e.g., 7123456789:AAHxyzABC123...) into Dashboard *Bot Token* field.
3. Get Chat ID	Determine destination user or channel ID	• Personal: Start bot, search **@userinfobot**, send /start to copy ID. • Locality Channel: Add bot as Admin to your Telegram channel, send a post, and check ID with **@RawDataBot** (starts with -100...).
4. Connect	Activate live dispatch in Dashboard	Click **'Save & Connect Telegram'** or test with **'Send Test Alert Handshake'**.

7. Operational Verification Checklist

[ ] Host Server Verification: Terminal shows *Uvicorn running on http://0.0.0.0:8000*.
[ ] Dashboard UI Check: Open http://localhost:8000 in browser; tactical map displays.
[ ] Hardware Antenna Check: GPS patch antenna oriented flat-face upwards with clear view of sky.

[ ] Serial Monitor Check: Arduino Serial Monitor at 115200 baud displays *[DASHBOARD] Sent! Server code: 200*.
[ ] Real-Time Plot Check: Dynamic marker (■, ■, ■■) flies to coordinates on OpenStreetMap.